

Códigos Python Gerados pela GEMINI e CLAUDE

Telemetria de Voo: Conversão de Tempo

Claude:

```
t = int(input("Informe o tempo em segundos: "))

dias      = t // 86400
resto1    = t % 86400
horas     = resto1 // 3600
resto2    = resto1 % 3600
minutos   = resto2 // 60
segundos  = resto2 % 60

print(f"{dias} dia(s), {horas}:{minutos:02d}:{segundos:02d}")
```

Gemini:

```
# --- Algoritmo Conversão Tempo ---
def conversao_tempo():
    t = int(input("Digite o total de segundos (>= 0): "))

    dias = t // 86400
    resto1 = t % 86400
    horas = resto1 // 3600
    resto2 = resto1 % 3600
    minutos = resto2 // 60
    segundos = resto2 % 60

    print(f"{dias} dia(s), {horas}:{minutos}:{segundos}")
```

Ponte de Wheatstone

Claude:

```

R1    = float(input("R1 (Ω): "))
R2    = float(input("R2 (Ω): "))
R3    = float(input("R3 (Ω): "))
Vfonte = float(input("Tensão da fonte (V): "))

if R1 == 0:
    print("Erro: R1 não pode ser zero (divisão por zero).")
else:
    Rx    = R3 * R2 / R1
    R_total = (R1 + Rx) * (R2 + R3) / (R1 + Rx + R2 + R3)
    I_total = Vfonte / R_total
    P_total = Vfonte * I_total

    print(f"Rx = {Rx:.2f} Ω")
    print(f"R_total = {R_total:.2f} Ω")
    print(f"I_total = {I_total:.4f} A")
    print(f"P_total = {P_total:.4f} W")

    if round(R1 * Rx, 6) == round(R2 * R3, 6):
        print("Verificação: ponte EQUILIBRADA ✓")
    else:
        print("Verificação: ponte DESEQUILIBRADA")

```

Gemini:

```

# --- Algoritmo Wheatstone ---
def wheatstone():
    r1 = float(input("R1: "))
    r2 = float(input("R2: "))
    r3 = float(input("R3: "))
    v_fonte = float(input("V_fonte: "))

    if r1 == 0:
        print("Erro: R1 não pode ser zero")
    else:
        rx = r3 * r2 / r1
        # Cálculo da resistência total (Paralelo de séries)
        r_total = ((r1 + rx) * (r2 + r3)) / (r1 + rx + r2 + r3)
        i_total = v_fonte / r_total
        p_total = v_fonte * i_total

        print(f"Rx = {rx} Ω")
        print(f"I total = {i_total} A")
        print(f"P total = {p_total} W")

        if r1 * rx == r2 * r3:
            print("Ponte equilibrada")
        else:
            print("Ponte desequilibrada")

```

Nota Fiscal: Tributos em Cascata

Claude:

```

preco_fabrica = float(input("Preço de fábrica (R$): "))
aliq_ipi      = float(input("Alíquota IPI (%): "))
aliq_icms     = float(input("Alíquota ICMS (%): "))
aliq_pis_cofins = float(input("Alíquota PIS/COFINS (%): "))

ipi          = preco_fabrica * aliq_ipi / 100
base_icms    = preco_fabrica + ipi
icms         = base_icms * aliq_icms / 100
base_pc      = base_icms + icms
pis_cofins   = base_pc * aliq_pis_cofins / 100
preco_final  = base_pc + pis_cofins
carga_pct    = (preco_final - preco_fabrica) / preco_fabrica * 100

print(f"{'Preço de fábrica:':<25} R$ {preco_fabrica:>8.2f}")
print(f"{'IPI (' + str(aliq_ipi) + '%)':<25} R$ {ipi:>8.2f}")
print(f"{'Base ICMS:':<25} R$ {base_icms:>8.2f}")
print(f"{'ICMS (' + str(aliq_icms) + '%)':<25} R$ {icms:>8.2f}")
print(f"{'Base PIS/COFINS:':<25} R$ {base_pc:>8.2f}")
print(f"{'PIS/COFINS (' + str(aliq_pis_cofins) + '%)':<25} R$ {pis_cofins:>8.2f}")
print("-" * 38)
print(f"{'Preço final:':<25} R$ {preco_final:>8.2f}")
print(f"{'Carga tributária total:':<25} {carga_pct:>7.2f}%")

```

Gemini:

```

def nota_fiscal():
    preco_fabrica = float(input("Preço de fábrica: "))
    aliq_ipi = float(input("Alíquota IPI (%): "))
    aliq_icms = float(input("Alíquota ICMS (%): "))
    aliq_pis_cofins = float(input("Alíquota PIS/COFINS (%): "))

    ipi = preco_fabrica * aliq_ipi / 100
    base_icms = preco_fabrica + ipi
    icms = base_icms * aliq_icms / 100
    base_pc = base_icms + icms
    pis_cofins = base_pc * aliq_pis_cofins / 100

    preco_final = base_pc + pis_cofins
    carga_pct = (preco_final - preco_fabrica) / preco_fabrica * 100

    print(f"Preço de fábrica: R$ {preco_fabrica:.2f}")
    print(f"IPI: R$ {ipi:.2f}")
    print(f"ICMS: R$ {icms:.2f}")
    print(f"PIS/COFINS: R$ {pis_cofins:.2f}")
    print(f"Preço final: R$ {preco_final:.2f}")
    print(f"Carga total: {carga_pct:.2f}%")

```

Laudo Geotécnico

Claude:

```
spt      = int(input("SPT (nº de golpes): "))
cap_carga = float(input("Capacidade de carga (kPa): "))

if spt < 5 or cap_carga < 80:
    print("INAPTO: solo muito fraco – reforço necessário")
elif spt < 10 or cap_carga < 150:
    print("RESTRITO: fundação especial recomendada")
    if spt < 10:
        print(f" → SPT no limite ({spt})")
    if cap_carga < 150:
        print(f" → Capacidade de carga no limite ({cap_carga} kPa)")
elif spt >= 30 and cap_carga >= 300:
    print("EXCELENTE: solo de alta capacidade")
else:
    print("ADEQUADO: fundação convencional viável")
```

Gemini:

```
def laudo_geotecnico():
    spt = float(input("SPT: "))
    cap_carga = float(input("Capacidade de carga: "))

    if spt < 5 or cap_carga < 80:
        print("Insuficiente: solo muito fraco - reforço necessário")
    elif spt < 10 or cap_carga < 150:
        print("Restrito: fundação especial recomendada")

    if spt < 10:
        print("-> SPT no limite")

    if cap_carga < 150:
        print("-> Capacidade de carga no limite")

    if spt >= 30 and cap_carga >= 300:
        print("Excelente: solo de alta capacidade")
    else:
        print("Adequado: fundação convencional viável")
```

Inspeção de Solda e Velocidade de Esteira

Claude:

```

print("=== PARTE A – Inspeção de Solda ===")
nominal = float(input("Resistência nominal (MPa): "))
medida = float(input("Resistência medida (MPa): "))

tolerancia_abs = nominal * 0.08
desvio = medida - nominal

if -tolerancia_abs <= desvio <= tolerancia_abs:
    print("SOLDA APROVADA")
else:
    desvio_pct = desvio / nominal * 100
    print(f"SOLDA REPROVADA")
    print(f"Desvio: {desvio:+.2f} MPa ({desvio_pct:+.2f}%)")

print("\n=== PARTE B – Velocidade da Esteira ===")
alvo = float(input("Velocidade alvo (m/min): "))
atual = float(input("Velocidade atual (m/min): "))

if atual > alvo * 1.10:
    print("VELOCIDADE ALTA – risco de posicionamento incorreto")
elif atual < alvo * 0.90:
    print("VELOCIDADE BAIXA – gargalo na produção")
else:
    print("VELOCIDADE OK")

```

Gemini:

```

def inspecao():
    # Parte A - Solos
    resistencia_nominal = float(input("Resistência nominal: "))
    resistencia_medida = float(input("Resistência medida: "))

    tolerancia = resistencia_nominal * 0.08
    desvio = resistencia_medida - resistencia_nominal

    if desvio >= -tolerancia and desvio <= tolerancia:
        print("Solo Aprovado")
    else:
        desvio_pct = (desvio / resistencia_nominal) * 100
        print("Solo Reprovado")
        print(f"Desvio: {desvio} (Medido: {desvio_pct:.2f}%)")

    # Parte B - Esteira
    vel_alvo = float(input("Velocidade alvo: "))
    vel_atual = float(input("Velocidade atual: "))

    if vel_atual > vel_alvo * 1.10:
        print("Velocidade Alta - risco de processamento incorreto")
    elif vel_atual < vel_alvo * 0.90:
        print("Velocidade Baixa - gargalo na produção")
    else:
        print("Velocidade OK")

```